

Seminar Presentation

전자서명 × 하이브리드 스키마 × 무결성 로그

저장-기록-검증의 완전한 사이클

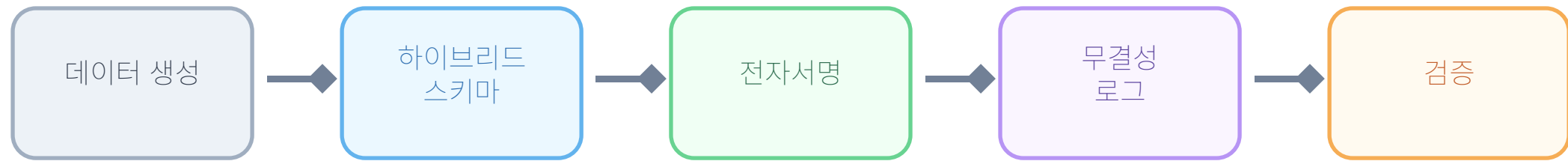
발표자 이승원

일시 2026년 4월

YOUTUBE <https://youtu.be/ODN7dAN9M4I>



전체 흐름 요약 – 저장·기록·검증의 선형 흐름



각 단계의 핵심 역할

[저장] 하이브리드 스키마

정형 데이터와 반정형 데이터(서명 메타데이터, BLOB 서명)를 유연하게 분리 저장

[기록] 무결성 로그

모든 변경 이력을 해시 체인으로 엮고 증거를 시간 순서대로 안전하게 기록

[검증] 전자서명 + 해시체인

시간의 흐름이 담긴 해시 체인과 전자서명의 유효성을 대조하여 위변조 여부 확인



하이브리드 스키마의 역할 – 저장 단계

[개념] 정형(Structured) 컬럼과 반정형(JSON/BLOB) 컬럼을 한 설계에 공존시키는 방법

서명 알고리즘(RSA, Dilithium 등)이 레코드마다 다를 수 있어 고정 컬럼으로는 다형성을 수용할 수 없음 → JSON 컬럼으로 유연성 확보

records (메인 테이블)			
id	BIGINT	정형	PK, 식별자
payload	TEXT	정형	실제 비즈니스 데이터
created_at	DATETIME	정형	생성 시각
sig_meta	JSON	반정형	서명 메타데이터 { "alg": "RSA-2048", "key_id": "k-001" }

1:1 관계 (FK)

signatures (서명 분리 테이블)			
record_id	BIGINT	정형	FK → records.id
signature	BLOB	반정형	실제 서명 바이트열 가변 크기 (수백~수천 B)

서명 데이터(BLOB) 분리 이유

서명 바이트 크기가 매우 클 수 있어 같은 row에 두면 테이블 폴 스캔 성능이 저하됨. 메인 테이블을 경량으로 유지.

Trade-offs: 장점: 알고리즘 다형성 수용, 메인 테이블 경량화 / 단점: JOIN 발생 쿼리 복잡도 증가, JSON 인덱싱 제약



전자서명 메커니즘 – 서명 생성 단계 (애플리케이션 레이어)

1. payload 입력

"주문#A" (원문 데이터)

2. 해시 계산

hash = SHA-256(payload) → e3b0c442...

고정 크기(32 bytes) 변환으로 속도 확보

3. 서명 생성

signature = Sign(hash, 개인키)

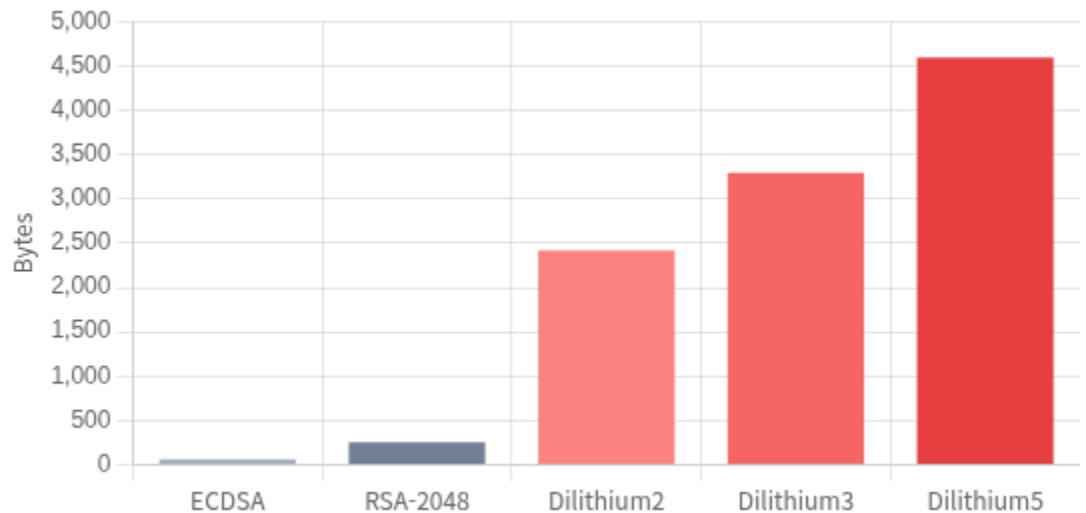
개인키 소유자만 생성 가능 (위변조 불가 증거)

4. DB 저장 (메타와 바이트 분리)

records.sig_meta ← {alg, key_id, signed_at}

signatures.signature ← [서명 바이트열]

알고리즘별 서명 크기 비교 (Bytes)



시사점

양자내성암호(PQC) 서명은 기존 RSA 대비 10~18배 큼니다. 레코드마다 서명하면 스토리지가 폭증하므로 배치 단위 서명 전략이 필수적입니다.

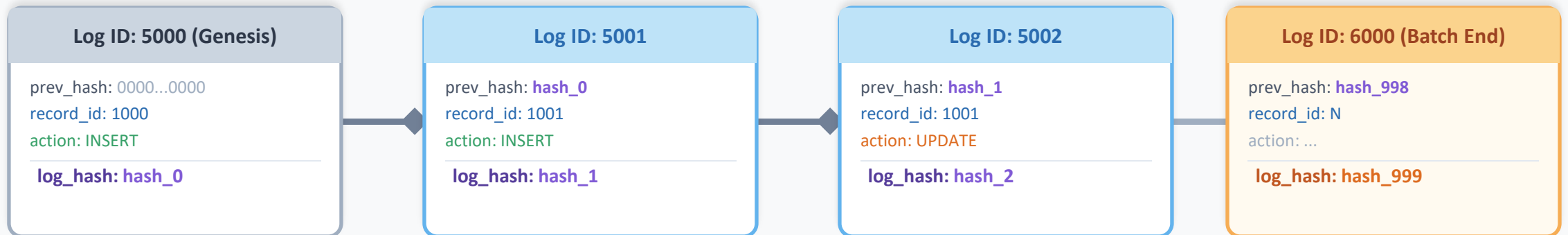


무결성 로그 – 해시 체인

[개념] 데이터 변경(INSERT/UPDATE/DELETE)의 사실과 증거를 순서대로 기록하는 감사 로그

※ MySQL 내장 로그(binlog 등)에는 암호학적 서명이 없음 → 무결성 보장은 애플리케이션 레벨 감사 로그(audit_log)에서 담당

$$\text{log_hash}[N] = \text{SHA-256}(\text{prev_hash} + \text{record_id} + \text{action} + \text{timestamp})$$



변조 탐지 원리

중간 로그(예: Log 5001)의 데이터를 변조하면, 해당 노드의 log_hash 값이 달라지며, 이는 연쇄적으로 다음 노드(Log 5002)의 prev_hash 계산을 실패하게 만들어 변조 즉시 탐지가 가능합니다.



검증 단계 – 세 요소가 합쳐지는 순간

특정 레코드가 변조됐는지 검증하는 전체 절차 및 시나리오별 탐지 지점

1. 로그 조회

audit_log에서 해당 record_id의 log_hash 조회

2. 해시 체인 재계산

log_hash[N-1]부터 체인 재계산 → 현재 hash 일치 확인

3. 서명 검증

해당 signature를 공개키로 검증

4. 공개키 특정 (Verify)

sig_meta의 key_id로 공개키 특정 후 서명 최종 Verify

5. payload 해시 대조

records 테이블의 payload 해시 재계산 → 메타와 대조

변조 시나리오별 탐지 지점 매핑

5단계 탐지 payload만 변경 시도

원문이 바뀌면 5단계에서 payload 해시가 불일치하여 적발됨

2단계 탐지 audit_log 변경 시도

로그를 조작하면 2단계에서 해시 체인이 붕괴되어 연쇄 적발됨

3단계 탐지 signature 변경 시도

서명 자체를 조작하면 3단계에서 공개키 검증(Verify)이 실패함

원천 차단 세 곳 동시 변경 시도

모두 조작하려 해도 개인키 없이는 유효한 서명을 새로 생성할 수 없어 사실상 불가능함



세 요소의 역할 정리 – 역할과 담당 단계 매핑

하이브리드 스키마

담당: 저장 (Storage)

데이터를 담는 "그릇"

정형 + 반정형 공존: 정형 데이터와 서명 메타 (JSON) 및 바이트(BLOB)를 통합 수용

다형성 확보: RSA, Dilithium 등 크기가 다양한 알고리즘을 유연하게 저장

성능 최적화: BLOB 테이블 분리로 메인 테이블 풀 스캔 성능 유지

전자서명

담당: 증거 생성 (Evidence)

위변조 불가 "증거"

암호학적 보장: payload 해시값에 서명하여 변경 불가능한 증거 생성

PQC 대비: 양자 컴퓨터 공격에 대비한 Dilithium 등 차세대 알고리즘 적용 지원

비인가 변경 차단: 개인키가 없는 공격자의 임의 데이터 조작을 원천 봉쇄

무결성 로그

담당: 기록/검증 (Chain)

시간 순으로 묶는 "체인"

해시 체인 구조: 변경 이력을 이전 상태 (prev_hash)와 엮어 연쇄적인 무결성 보장

즉각적 탐지: 중간 로그나 데이터 변조 시 후속 해시가 어긋나 변조 사실 즉시 적발

그릇(하이브리드 스키마) + 증거(전자서명) + 체인(무결성 로그) = 완전한 데이터 무결성 보장 사이클 완성

결론 — 저장-기록-검증의 완전한 사이클

그릇(하이브리드 스키마) + 증거(전자서명) + 체인(무결성 로그)
= 완전한 데이터 무결성 보장 사이클 완성

MySQL

트랜잭션과 ACID 특성을 기반으로 애플리케이션이 신뢰할 수 있는 견고한 저장소 환경을 제공합니다.

하이브리드 스키마

정형 데이터와 반정형 (JSON/BLOB) 컬럼의 공존으로 과도기적 알고리즘의 서명 다형성을 유연하게 수용합니다.

전자서명

데이터 원문 해시를 기반으로 생성되며, 사후 임의 변경이 수학적으로 불가능한 강력한 증거를 제공합니다.

무결성 로그

해시 체인 구조를 이용하여 변경 이력 전체를 보호합니다.

